

Lecture 27

Compiler Design CSC-305

Awanish Pandey

Department of Computer Science and Engineering
Indian Institute of Technology
Roorkee

October 6, 2026

Syntax directed translation of boolean expressions

- $E \rightarrow id_1$ relop id_2

Syntax directed translation of boolean expressions

- $E \rightarrow id_1$ relop id_2
  $E.place = newtmp()$
  emit(if $id_1.place$ relop $id_2.place$ goto state+3)
  $emit(E.place = 0)$
  $emit(goto\ state + 2)$
  $emit(E.place = 1)$

Syntax directed translation of boolean expressions

- $E \rightarrow id_1$ relop id_2
  $E.place = newtmp()$
  emit(if $id_1.place$ relop $id_2.place$ goto state+3)
  $emit(E.place = 0)$
  $emit(goto\ state + 2)$
  $emit(E.place = 1)$
- $E \rightarrow true$

Syntax directed translation of boolean expressions

- $E \rightarrow id_1$ relop id_2
  $E.place = newtmp()$
  emit(if $id_1.place$ relop $id_2.place$ goto state+3)
  $emit(E.place = 0)$
  $emit(goto\ state + 2)$
  $emit(E.place = 1)$
- $E \rightarrow true$
  E.place = newtmp()
  emit(E.place = '1')

Syntax directed translation of boolean expressions

- $E \rightarrow id_1$ relop id_2
  $E.place = newtmp()$
  emit(if $id_1.place$ relop $id_2.place$ goto state+3)
  $emit(E.place = 0)$
  $emit(goto\ state + 2)$
  $emit(E.place = 1)$
- $E \rightarrow true$
  E.place = newtmp()
  emit(E.place = '1')
- $E \rightarrow false$

Syntax directed translation of boolean expressions

- $E \rightarrow id_1$ relop id_2
  $E.place = newtmp()$
  emit(if $id_1.place$ relop $id_2.place$ goto state+3)
  $emit(E.place = 0)$
  $emit(goto\ state + 2)$
  $emit(E.place = 1)$
- $E \rightarrow true$
  E.place = newtmp()
  emit(E.place = '1')
- $E \rightarrow false$
  E.place = newtmp()
  emit(E.place = '0')

Syntax directed translation of boolean expressions

- $E \rightarrow id_1$ relop id_2
  $E.place = newtmp()$
  emit(if $id_1.place$ relop $id_2.place$ goto state+3)
  $emit(E.place = 0)$
  $emit(goto\ state + 2)$
  $emit(E.place = 1)$
- $E \rightarrow true$
  E.place = newtmp()
  emit(E.place = '1')
- $E \rightarrow false$
  E.place = newtmp()
  emit(E.place = '0')

Write 3AC Code for `a < b or c < d and e < f`

Short Circuit Evaluation of boolean expressions

- Translate boolean expressions without

Short Circuit Evaluation of boolean expressions

- Translate boolean expressions without
  - generating code for boolean operators

Short Circuit Evaluation of boolean expressions

- Translate boolean expressions without
  - generating code for boolean operators
  - evaluating the entire expression
- Flow if control statements

Short Circuit Evaluation of boolean expressions

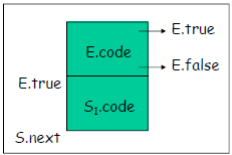
- Translate boolean expressions without
  - generating code for boolean operators
  - evaluating the entire expression
- Flow if control statements

  $S \rightarrow$ if E then S_1

  | if E then S_1 else S_2

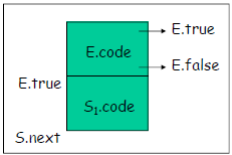
  | while E do S_1

If-then



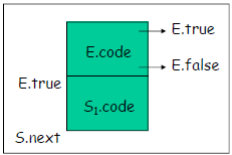
$S \rightarrow$ if E then S_1
$E.true = newlabel()$

If-then



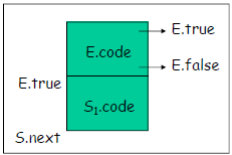
$S \rightarrow$ if E then S_1
$E.true = newlabel()$
$E.false = S.next$

If-then



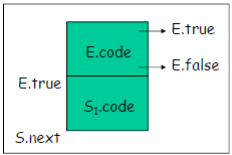
$S \rightarrow$ if E then S_1
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.next$

If-then



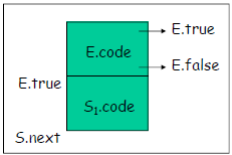
$S \rightarrow$ if E then S_1
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.next$
$S.code = E.code||$

If-then



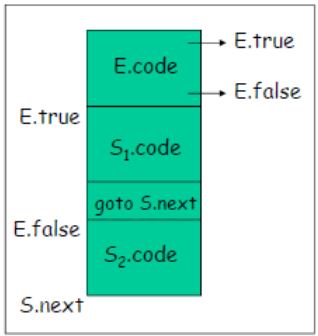
$S \rightarrow$ if E then S_1
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.next$
$S.code = E.code||$
$gen(E.true \text{“ : ”})||$

If-then

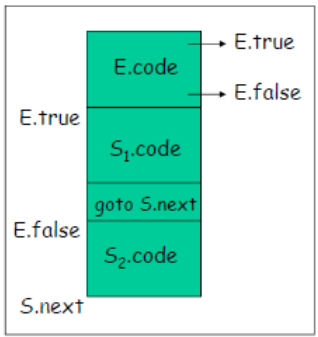


$S \rightarrow$ if E then S_1
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.next$
$S.code = E.code||$
$gen(E.true\text{“ : ”})||$
$S1.code$

If-else

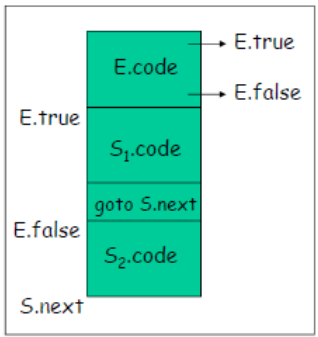

E.code
E.true
E.false
E.true
S_1.code
goto S.next
E.false
S_2.code
S.next

If-else



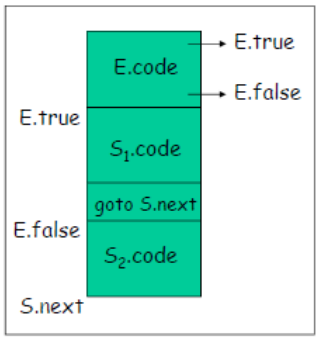
$S \rightarrow$ if E then S_1 else S_2

If-else



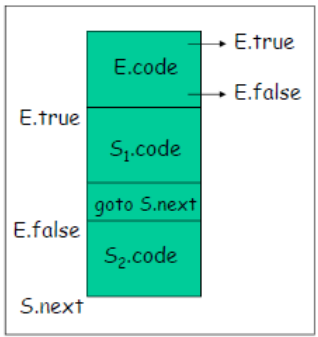
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$

If-else



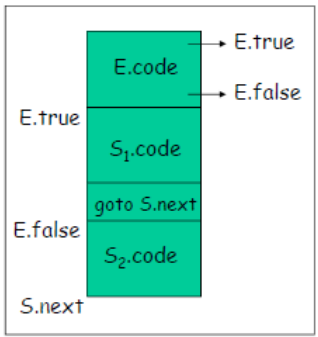
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$

If-else



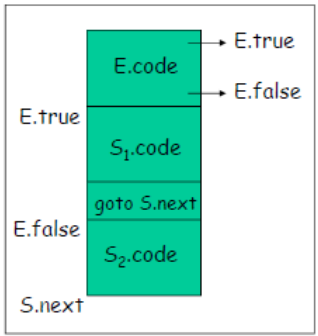
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$

If-else



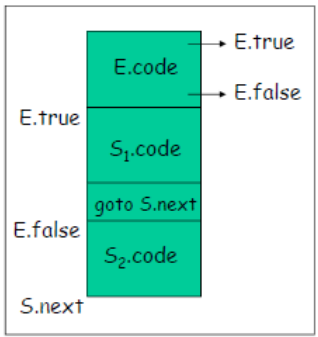
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$
$S_2.next = S.next$

If-else



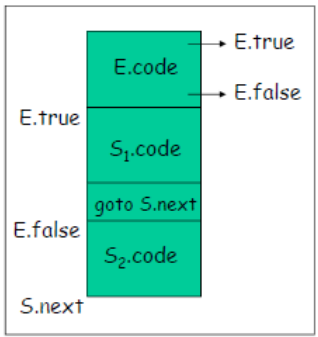
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$
$S_2.next = S.next$
$S.code = E.code||$

If-else



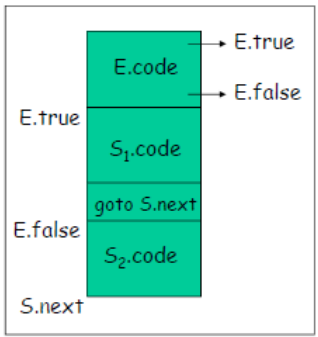
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$
$S_2.next = S.next$
$S.code = E.code||$
$gen(E.true\text{“ : ”})||$

If-else



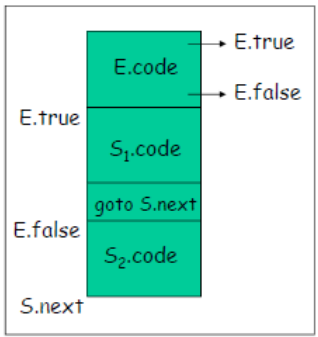
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$
$S_2.next = S.next$
$S.code = E.code||$
$gen(E.true \text{“ : ”})||$
$S_1.code||$

If-else



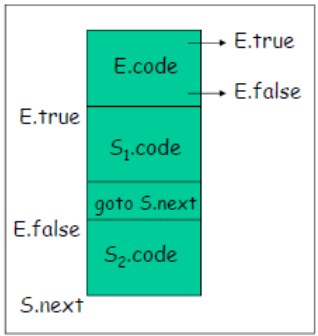
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$
$S_2.next = S.next$
$S.code = E.code||$
$gen(E.true\text{“ : ”})||$
$S_1.code||$
$gen(goto\ S.next)||$

If-else



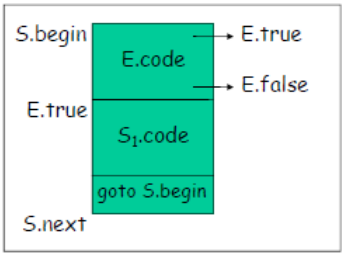
$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$
$S_2.next = S.next$
$S.code = E.code||$
$gen(E.true\text{“ : ”})||$
$S_1.code||$
$gen(goto\ S.next)||$
$gen(E.false\text{“ : ”})||$

If-else

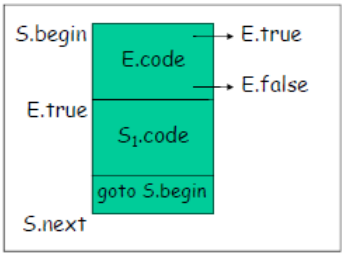


$S \rightarrow$ if E then S_1 else S_2
$E.true = newlabel()$
$E.false = newlabel()$
$S_1.next = S.next$
$S_2.next = S.next$
$S.code = E.code||$
$gen(E.true \text{“ : ”})||$
$S_1.code||$
$gen(goto\ S.next)||$
$gen(E.false \text{“ : ”})||$
$S_2.code$

While

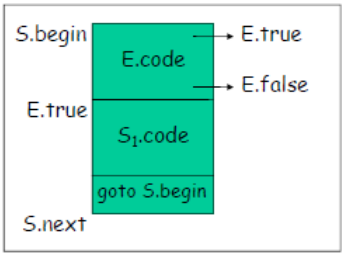


While



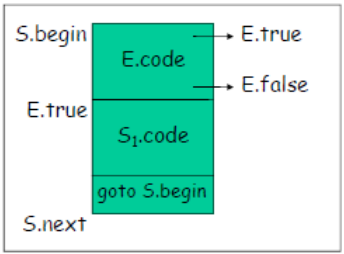
$S \rightarrow$ while E do S_1

While



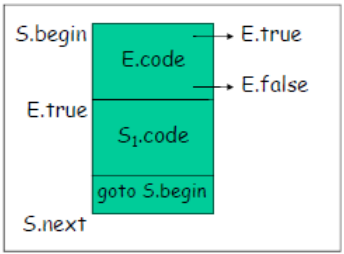
$S \rightarrow$ while E do S_1
$S.begin = newlabel()$

While



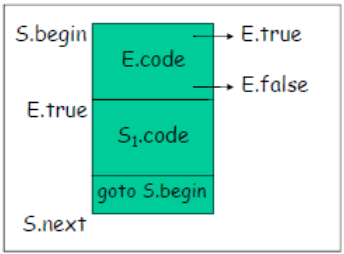
$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$

While



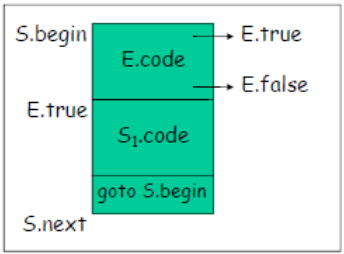
$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$
$E.false = S.next$

While



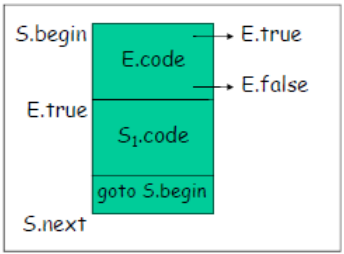
$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.begin$

While



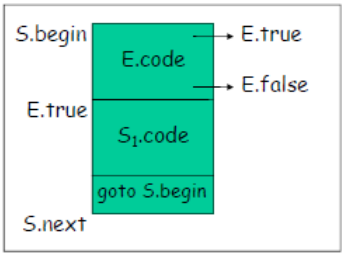
$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.begin$
$S.code = gen(S.begin' :')||$

While



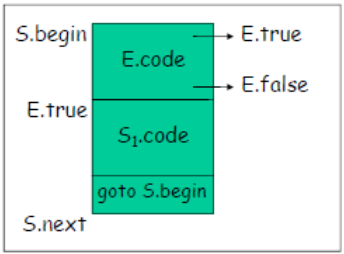
$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.begin$
$S.code = gen(S.begin' :')||$
$E.code||$

While



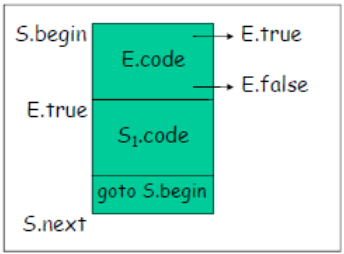
$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.begin$
$S.code = gen(S.begin' :')||$
$E.code||$
$gen(E.true' :')||$

While



$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.begin$
$S.code = gen(S.begin' :')||$
$E.code||$
$gen(E.true' :')||$
$S_1.code||$

While



$S \rightarrow$ while E do S_1
$S.begin = newlabel()$
$E.true = newlabel()$
$E.false = S.next$
$S_1.next = S.begin$
$S.code = gen(S.begin' :')||$
$E.code||$
$gen(E.true' :')||$
$S_1.code||$
$gen(gotoS.begin)$

Control flow translation of boolean expression

- $E \rightarrow E_1$ or E_2

Control flow translation of boolean expression

- $E \rightarrow E_1$ or E_2 $\quad E_1.true = E.true$
  $E_1.false = \text{newlabel}()$
  $E_2.true = \text{E.true}$
  $E_2.false = \text{E.false}$
  $\text{E.code} = E_1.\text{code} \mid\mid \text{gen}(E_1.\text{false})$
  $\mid\mid E_2.\text{code}$

Control flow translation of boolean expression

- $E \rightarrow E_1$ or E_2 $\quad E_1.true = E.true$
  $E_1.false = \text{newlabel}()$
  $E_2.true = \text{E.true}$
  $E_2.false = \text{E.false}$
  $\text{E.code} = E_1.\text{code} \mid\mid \text{gen}(E_1.\text{false})$
  $\mid\mid E_2.\text{code}$
- $E \rightarrow E_1$ and E_2

Control flow translation of boolean expression

- $E \rightarrow E_1$ or E_2 $\quad E_1.true = E.true$
  $E_1.false = \text{newlabel}()$
  $E_2.true = \text{E.true}$
  $E_2.false = \text{E.false}$
  $\text{E.code} = E_1.\text{code} \mid\mid \text{gen}(E_1.\text{false}) \mid\mid E_2.\text{code}$
- $E \rightarrow E_1$ and E_2 $\quad E_1.\text{true} = \text{newlabel}()$
  $E_1.\text{false} = \text{E.false}$
  $E_2.\text{true} = \text{E.true}$
  $E_2.\text{false} = \text{E.false}$
  $\text{E.code} = E_1.\text{code} \mid\mid \text{gen}(E_1.\text{true}) \mid\mid E_2.\text{code}$

Control flow translation of boolean expression

- E $\rightarrow$ not E_1 $\quad$ E_1.true $=$ E.false

Control flow translation of boolean expression

- E $\rightarrow$ not E_1 $\quad$ E_1.true $=$ E.false
  E_1.false $=$ E.true
  E.code $=$ E_1.code

Control flow translation of boolean expression

- E $\rightarrow$ not E_1 $\quad$ E_1.true = E.false
  E_1.false = E.true
  E.code = E_1.code
- E $\rightarrow$ (E_1) $\quad$ E_1.true = E.true
  E_1.false = E.false

Control flow translation of boolean expression

- E $\rightarrow$ not E_1 $\quad$ E_1.true = E.false
  E_1.false = E.true
  E.code = E_1.code
- E $\rightarrow$ (E_1) $\quad$ E_1.true = E.true
  E_1.false = E.false
  E.code = E_1.code
- E $\rightarrow$ id_1 relop id_2

Control flow translation of boolean expression

- E $\rightarrow$ not E_1 $\quad$ E_1.true = E.false
  E_1.false = E.true
  E.code = E_1.code
- E $\rightarrow$ (E_1) $\quad$ E_1.true = E.true
  E_1.false = E.false
  E.code = E_1.code
- E $\rightarrow$ id_1 relop id_2
  E.code = gen(if id_1 relop id_2 goto E.true) || gen(goto E.false)

Control flow translation of boolean expression

- E $\rightarrow$ not E_1 $\quad$ E_1.true = E.false
  E_1.false = E.true
  E.code = E_1.code
- E $\rightarrow$ (E_1) $\quad$ E_1.true = E.true
  E_1.false = E.false
  E.code = E_1.code
- E $\rightarrow$ id_1 relop id_2
  E.code = gen(if id_1 relop id_2 goto E.true) || gen(goto E.false)
- E $\rightarrow$ `true` $\quad$ E.code = gen(goto E.true)

Control flow translation of boolean expression

- E $\rightarrow$ not E_1 $\quad$ E_1.true = E.false
  E_1.false = E.true
  E.code = E_1.code
- E $\rightarrow$ (E_1) $\quad$ E_1.true = E.true
  E_1.false = E.false
  E.code = E_1.code
- E $\rightarrow$ id_1 relop id_2
  E.code = gen(if id_1 relop id_2 goto E.true) || gen(goto E.false)
- E $\rightarrow$ `true` $\quad$ E.code = gen(goto E.true)
- E $\rightarrow$ `false` $\quad$ E.code = gen(goto E.false)

Control flow translation of boolean expression

- $E \rightarrow E_1$ or E_2 $\quad E_1.true = E.true$
  $E_1.false = \text{newlabel}()$
  $E_2.true = \text{E.true}$
  $E_2.false = \text{E.false}$
  $\text{E.code} = E_1.\text{code} \mid\mid \text{gen}(E_1.\text{false}) \mid\mid E_2.\text{code}$
- $E \rightarrow E_1$ and E_2 $\quad E_1.\text{true} = \text{newlabel}()$
  $E_1.\text{false} = \text{E.false}$
  $E_2.\text{true} = \text{E.true}$
  $E_2.\text{false} = \text{E.false}$
  $\text{E.code} = E_1.\text{code} \mid\mid \text{gen}(E_1.\text{true}) \mid\mid E_2.\text{code}$

BackPatching

- Way to implement boolean expressions and flow of control statements in one pass

BackPatching

- Way to implement boolean expressions and flow of control statements in one pass
- Labels are indices into this array

BackPatching

- Way to implement boolean expressions and flow of control statements in one pass
- Labels are indices into this array
- **makelist(i):** create a newlist containing only i, return a pointer to the list.

BackPatching

- Way to implement boolean expressions and flow of control statements in one pass
- Labels are indices into this array
- **makelist(i):** create a newlist containing only i, return a pointer to the list.
- **merge(p1,p2):** merge lists pointed to by p1 and p2 and return a pointer to the concatenated list

BackPatching

- Way to implement boolean expressions and flow of control statements in one pass
- Labels are indices into this array
- **makelist(i):** create a newlist containing only i, return a pointer to the list.
- **merge(p1,p2):** merge lists pointed to by p1 and p2 and return a pointer to the concatenated list
- **backpatch(p,i):** insert i as the target label for the statements in the list pointed to by p